

Relatório de Auditoria de Segurança

ThermoProActive / Pred Ativos

Emitido em 20 de setembro de 2026 · revisado em 21 de setembro de 2026

revisão auditada: cac2513 + ajustes de 21/09

1

CRÍTICA

2

ALTA

1

MÉDIA

3

BAIXA

ESCOPO AUDITADO

- backend/ — Django 5.1 + Django REST Framework (105 módulos Python; .env e migrations fora do escopo)
- frontend/ — Next.js 15 (App Router) + React 19 + TypeScript (src/ completo)
- Deploy — docker-compose.yml, docker-compose.prod.yml, docker-compose.http.yml, backend/Dockerfile, frontend/Dockerfile, deploy/nginx/*.conf
- Segredos — .gitignore, .env.prod.example, backend/.env.example, frontend/.env.local.example e os 138 commits do histórico do repositório

NOTA METODOLÓGICA

As cinco categorias do pedido foram mapeadas para os equivalentes da stack detectada. O mapeamento completo está na seção 1; o resumo é: não há RLS neste projeto, o isolamento de inquilino é filtro manual no ORM; a matriz de papéis do frontend foi cruzada uma a uma com as permission_classes do DRF; todos os handlers de rota foram percorridos, não amostras; e as categorias sem equivalente na stack estão declaradas como não aplicáveis em vez de preenchidas à força.

1. Escopo e nota metodológica

Categoria / item	Como foi mapeada para esta stack
Stack detectada	Backend Django 5.1 + DRF, ORM do Django (sem SQL cru), autenticação JWT por cookie HttpOnly (SimpleJWT + CSRF + sessão materializada no banco). Frontend Next.js 15 App Router, React 19, sem biblioteca de sanitização de HTML. PostgreSQL 16 em produção, SQLite no dev. Deploy por Docker Compose atrás de Nginx. Não há Supabase, Helm, Terraform nem pipeline de CI no repositório.
1. Banco sem tranca	Não existe RLS: o isolamento de inquilino é feito por FILTRO MANUAL no ORM, em duas implementações irmãs — <code>escopo_cliente()</code> em <code>apps/coletas/views.py</code> e o atributo <code>campo_cliente</code> de <code>BaseCadastroViewSet</code> em <code>apps/cadastros/views.py</code> . A auditoria mapeou todo <code>ViewSet</code> que expõe dado de cliente e verificou se declara o filtro, se o filtro falha fechado e se existe contraparte de escrita (<code>impedir_cross_tenant</code>) no caminho de criação e de realocação.
2. Permissão no navegador	O front declara a matriz de papéis em <code>frontend/src/lib/permissions.ts</code> (função <code>can()</code> + <code>REGRAS_ROTA</code>). Cada habilidade foi cruzada com a <code>permission_class</code> DRF do endpoint correspondente (<code>apps/accounts/permissions.py</code>).
3. IDOR	Percorridos TODOS os handlers de rota do backend — 9 módulos de <code>urls.py</code> , 31 <code>ViewSets/APIViews</code> e cada <code>@action detail=True</code> — verificando se a busca do objeto passa por um <code>get_queryset()</code> escopado ou por um <code>get_object_or_404</code> solto.
4. Chaves expostas	Varredura de <code>settings</code> , <code>composes</code> , <code>Dockerfiles</code> , <code>scripts de seed</code> , <code>configs de Nginx</code> , arquivos de exemplo e do histórico git completo (<code>git rev-list --all</code>). Atenção especial a defaults públicos que viram segredo real e à ausência de validação de startup que os recuse.
5. Inputs sem tratamento (XSS)	Grep exaustivo no front por <code>dangerouslySetInnerHTML</code> , <code>innerHTML</code> , <code>eval</code> , <code>new Function</code> , <code>document.write</code> e por URLs controladas por usuário em <code>href/src</code> ; no back por <code>mark_safe</code> , <code>format_html</code> , <code>render_to_string</code> , templates HTML próprios e HTML em corpo de e-mail.

Categorias não aplicáveis a esta stack

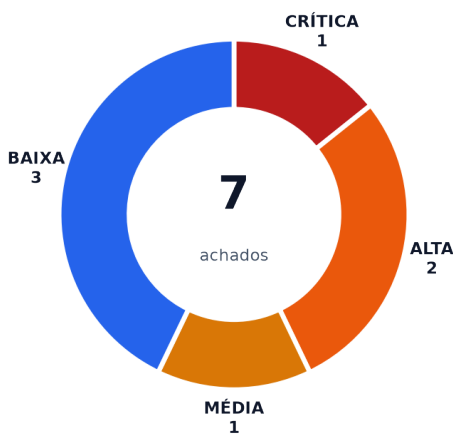
Item	Situação
5. Inputs sem tratamento (XSS)	Categoria verificada e SEM ACHADOS — nenhuma constatação foi forçada. O projeto não renderiza markdown nem HTML vindo do usuário, não há lib de sanitização no <code>package.json</code> porque não existe ponto que precise de uma, e o único <code>dangerouslySetInnerHTML</code> usa constante literal estática. Detalhes na tabela de verificação abaixo.
RLS / Supabase	Não se aplica: o projeto não usa Supabase nem Row Level Security do PostgreSQL. O equivalente na stack — filtro manual por cliente no ORM — foi auditado em profundidade na categoria 1.
CI / Helm / Terraform	Não se aplica: o repositório não tem <code>.github/workflows</code> , charts Helm nem módulos Terraform. O deploy é Docker Compose + Nginx, ambos auditados.

2. Resumo executivo

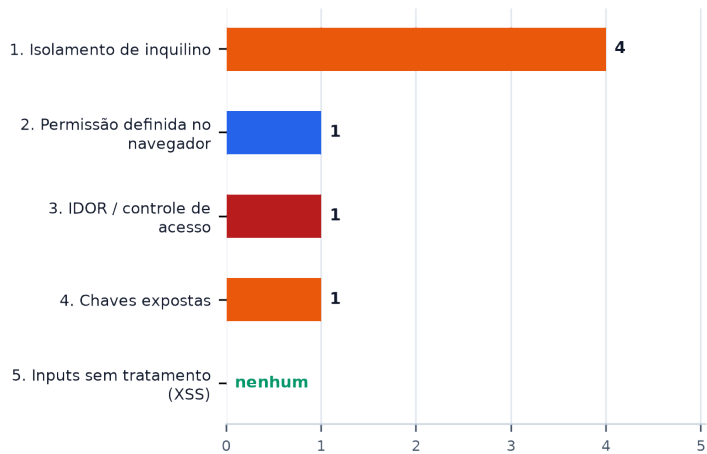
A auditoria produziu 7 achados verificados no código real — 1 crítico, 2 altos, 1 médio, 3 baixos — e 4 deles foram reproduzidos com prova de conceito executável contra a própria API (teste Django + APIClient), não apenas inferidos da leitura. O achado crítico é um par de endpoints JWT legados que continua publicado ao lado do fluxo de autenticação endurecido e anula MFA, bloqueio de conta e revogação de sessão. Em contrapartida, o núcleo do isolamento multi-tenant e a superfície de XSS estão sólidos — a seção 3 detalha o que foi verificado e está correto.

Achados de severidade alta: A-03 — /media/ é servido sem autenticação e expõe evidências técnicas e assinaturas digitais; A-07 — Segredos padrão embutidos no código, sem validação de startup que os recuse.

Revisão de 21 de setembro de 2026. O Portal do Cliente passou a ser somente consulta, com uma única escrita recortada (o retorno de informação da Ordem de Serviço). Isso fechou o vetor de A-02, que era ALTA e passou a BAIXA — a lacuna estrutural que o originou continua no código e está descrita no achado. Os demais achados não foram afetados.



Distribuição por severidade



Achados por categoria auditada

Severidade	Qtde.	Achados	Leitura
CRÍTICA	1	A-06	Corrigir antes do próximo deploy.
ALTA	2	A-03, A-07	Corrigir nesta sprint; há vazamento de dado ou de segredo.
MÉDIA	1	A-01	Corrigir na sequência; depende de um estado de configuração alcançável.
BAIXA	3	A-02, A-04, A-05	Dívida técnica de segurança; sem exploração conhecida hoje.

3. Pontos fortes e pontos fracos

Os pontos fortes abaixo não são elogios genéricos: cada um cita o arquivo e a linha que sustentam a afirmação, e juntos delimitam a cobertura desta auditoria — o que foi olhado e encontrado correto.

3.1 · O que está protegido (com evidência)

Controle verificado	Evidência no código
Filtro de tenant aplicado e com falha fechada nos 3 módulos principais	apps/coletas/views.py:42-53 e apps/cadastros/views.py:94-102 devolvem <code>qs.none()</code> quando o usuário é cliente e não tem vínculo. Declarado em todos os ViewSets de dado de cliente: <code>Cliente(id)</code> , <code>Área</code> , <code>Setor</code> , <code>Equipamento</code> , <code>Componente</code> , <code>DadosTecnicosMotor/Transformador</code> , <code>Rota</code> , <code>Inspeção</code> , as 3 <code>Medições</code> , <code>Relatório</code> , <code>Carregamento</code> , <code>ItemInspeção</code> , <code>Achado</code> , <code>AchadoImagem</code> , <code>Laudo</code> , <code>OrdemServiço</code> , <code>ServiçoCampo</code> , <code>BalanceamentoPlano/Ponto</code> e <code>EconomiaEnergética</code> .
Guardas de escrita cross-tenant presentes nos caminhos de criação e redistribuição	apps/coletas/views.py:56-79 (<code>impedir_cross_tenant</code>), apps/cadastros/views.py:117-139, apps/osp/views.py:29-46 e apps/coletas/serializers.py:158-173 (<code>AchadoSerializer.validate_item</code>) recusam apontar um registro para outro cliente — inclusive no caminho indireto pelo equipamento. A lacuna conhecida é o M2M de Rota (A-02), hoje sem vetor pelo Portal.
Portal reduzido a consulta, com a única escrita recortada campo a campo	Decisão de 2026-09-21: apps/accounts/permissions.py:69-110 (<code>InternoEditaClienteResponde</code>) libera ao cliente só PATCH/PUT, nunca POST nem DELETE, e apenas na Ordem de Serviço. QUAIS campos ele pode tocar está em apps/osp/serializers.py:6-45 (<code>CAMPOS_RETORNO_CLIENTE</code>); o <code>__init__</code> do serializer marca todo o resto como somente-leitura, então a trava não depende de o formulário esconder o campo. A FK de 'quem executou' tem o queryset recortado para a própria equipe, fechando o vazamento de nome entre inquilinos que o campo abriria. 22 testes em apps/osp/test_retorno_cliente.py e apps/coletas/test_dashboard_cliente.py prendem a regra.
Nenhum handler <code>detail=True</code> busca objeto fora do queryset escopado	Os 31 ViewSets/APIViews foram percorridos um a um: todo <code>@action detail=True</code> usa <code>self.get_object()</code> . Os três <code>get_object_or_404</code> do projeto são seguros — <code>laudos/views.py:34</code> está sob <code>IsInterno</code> (escopo global por desenho) e <code>servicos/atividades.py:105-108</code> e <code>:138</code> amarram o objeto ao carregamento já escopado.
Autenticação endurecida no fluxo que o frontend realmente usa	apps/accounts/auth_views.py:143-206: cookie <code>HttpOnly</code> + <code>SameSite=Lax</code> com path restrito, CSRF obrigatório (<code>authentication.py:11-21</code>), sessão materializada em <code>SessaoAutenticacao</code> e conferida a cada request, TOTP com janela de 1 passo e anti-replay via <code>ultimo_passo_usado</code> (<code>security.py:177-186</code>), códigos de recuperação com hash, bloqueio após 8 tentativas, throttles por escopo, rotação e blacklist de refresh. O problema é o endpoint legado paralelo (A-06), não este fluxo.
Tokens de uso único e convites tratados corretamente	apps/accounts/security.py:54-80: token opaco de 48 bytes, apenas o SHA-256 vai ao banco, invalidação dos anteriores ao emitir um novo, expiração curta e consumo único. Convites expiram em 72h e <code>CriarConviteSerializer.validate</code> (<code>auth_serializers.py:131-138</code>) impede o Master de um cliente convidar alguém com perfil interno — fechando a escalada de privilégio mais óbvia.
Nenhum segredo real no histórico do repositório	<code>.gitignore:7,9,10,18</code> cobre <code>backend/.env</code> , <code>.env.prod</code> , <code>.env*.local</code> e <code>frontend/.env.local</code> . A varredura de <code>git rev-list --all</code> nos 138 commits não encontrou nenhum <code>.env</code> , <code>.pem</code> ou chave privada. Os composes de produção usam <code>\${VAR}</code> sem default, e os <code>.example</code> só contêm placeholders.

Controle verificado	Evidência no código
Bundle do frontend não carrega segredo	frontend/Dockerfile:15-16 injeta apenas NEXT_PUBLIC_API_URL. frontend/src/lib/api.ts:1-4 documenta e implementa a decisão de nunca expor token ao JavaScript: a sessão vive só em cookie HttpOnly e o único valor que o JS manipula é o token CSRF.
Superfície de XSS praticamente nula	Um único dangerouslySetInnerHTML em todo o projeto (frontend/src/app/layout.tsx:89), com constante literal estática definida na linha 83. Zero eval/new Function/document.write. Nenhum markdown ou HTML renderizado a partir de dado do usuário. No backend, zero mark_safe/format_html/render_to_string, TEMPLATES.DIRS vazio e e-mails em texto puro (security.py:87-148).
Cabeçalhos e cookies de produção configurados com critério	backend/config/settings.py:220-235 aplica HSTS (só quando há TLS de fato), nosniff, X-Frame-Options DENY e Referrer-Policy quando DEBUG=False. COOKIES_SECURE (:158) é opt-out consciente e documentado para o cenário de VPS por IP, com o padrão seguro.
Banco não exposto e testes de escopo já existentes	docker-compose.prod.yml:28 deixa o PostgreSQL sem 'ports', só na rede interna do compose. apps/cadastros/test_escopo_cliente.py cobre 9 cenários de isolamento, incluindo test_cliente_sem_vinculo_ao_ve_nada e test_cliente_nao_acessa Equipamento de outro por id.

3.2 · Riscos centrais

Risco	Por quê
Existe uma porta de entrada paralela a toda a autenticação endurecida	Todo o investimento em MFA, bloqueio de conta, sessão revogável e CSRF fica sem efeito enquanto /api/auth/login/ continuar publicado. É o único achado crítico — e o de correção mais barata: duas linhas de urls.py.
O isolamento depende de o desenvolvedor lembrar de declarar o filtro, em quatro cópias diferentes	Não há RLS nem middleware de tenant: cada ViewSet precisa declarar campo_cliente ou chamar escopo_cliente à mão, e o helper foi copiado quatro vezes com semânticas divergentes. A-01 e A-04 são consequência direta disso. Um esquecimento não quebra nada visivelmente — só vaza.
A fronteira de inquilino para de existir na camada de arquivos	O Django protege as linhas do banco; o Nginx entrega as imagens. Evidências técnicas e assinaturas digitais ficam fora de qualquer controle de acesso (A-03).
Defaults de desenvolvimento são seguros por acaso, não por desenho	SECRET_KEY, DEBUG e ALLOWED_HOSTS têm default permissivo e nenhuma checagem de boot recusa esses valores em produção. O acerto atual depende de o operador não esquecer uma variável (A-07).
Guardas de escrita cobrem ForeignKey, mas não relações ManyToMany	O padrão impedir_cross_tenant só inspeciona o primeiro segmento do caminho ORM; qualquer M2M passa livre. Hoje isso atinge Rota.equipamentos (A-02), e o mesmo padrão se repetiria em qualquer M2M novo.
A superfície de escrita do Portal já mudou duas vezes em três dias	Só leitura até 18/09; parque inteiro liberado ao Master do cliente em 18-19/09; de volta a consulta em 21/09. A-02 era ALTA e virou BAIXA só por causa dessa última virada — nada no código impede a próxima. É o argumento mais forte a favor do teste de fumaça de autorização (P3): a regra precisa estar presa por teste, não por comentário.

3.3 · Categoria 5 (XSS): o que foi verificado e está correto

A categoria não gerou achados. A tabela registra cada ponto inspecionado, para que a ausência de achados seja auditável em vez de uma afirmação sem lastro.

Arquivo:linha	Vetor checado	Resultado
frontend/src/app/layout.tsx:83,89	dangerouslySetInnerHTML	OK — __html recebe INICIAR_TEMA, constante literal sem interpolação (linha 83).
frontend/src/app/carta/[id]/carta-client.tsx:72-73	innerHTML	OK — round-trip de DOM já renderizado pelo React para o paged.js; React escapa na escrita e a serialização preserva o escape.
frontend/src/app/imprimir/[id]/imprimir-client.tsx:80-81	innerHTML	OK — mesmo padrão do carta-client.
frontend/src/app/portal/page.tsx:244	href com dado do servidor	OK — passa por destinoNoPortal (permissions.ts:254-272), que só aceita prefixos de uma allowlist e cai em “/portal” fora dela.
frontend/src/features/alertas/lista-alertas.tsx:102	router.push(n.url) cru p/ interno	OK — Notificacao.url só é escrito por literais do servidor (notificacoes/signals.py:28,49) e o ViewSet é somente-leitura (notificacoes/views.py:11).
frontend/src – grep global	eval / new Function / document.write	OK — zero ocorrências.
backend/apps – grep global	mark_safe / format_html / render_to_string / templates .html	OK — zero ocorrências; TEMPLATES.DIRS vazio (settings.py:84).
backend/apps/accounts/security.py:87-148	HTML em e-mail	OK — send_mail com corpo em texto puro; nenhuma interpolação em HTML.
backend/apps/relatorios/views.py:80-81	Content-Disposition com dado de entrada	OK — “key” é validada contra o dicionário REPORTS antes de entrar no nome do arquivo (linhas 57-59).
backend/apps/coletas/views.py:629-635	Content-Disposition do carta-docx	OK — nome sanitizado caractere a caractere na linha 630.

4. Achados detalhados por categoria

Visão tabular exigida pelo escopo: severidade, arquivo:linha e descrição, agrupadas por categoria. O detalhamento técnico de cada achado — trecho de código, prova de conceito, condições de explorabilidade e correção — vem na seção 5.

Severidade	Arquivo:linha	Descrição
1. Isolamento de inquilino		
ALTA	deploy/nginx/app.conf.template:61-65 deploy/nginx/app.http.conf:21-25 backend/apps/coletas/models.py:614 backend/apps/accounts/models.py:108-110 backend/apps/cadastros/models.py:355 backend/apps/cadastros/models.py:418 backend/config/urls.py:33-34	A-03 — /media/ é servido sem autenticação e expõe evidências técnicas e assinaturas digitais Vazamento cross-tenant sem sequer precisar de conta: as fotos de termografia e vibração são a evidência técnica das anomalias de cada cliente (o que quebrou, onde, em que ativo), e as placas de identificação trazem fabricante, modelo e número de série do parque alheio. Pior, /media/assinaturas/ guarda a imagem da assinatura do responsável técnico usada para assinar laudos — baixá-la permite forjar um laudo com aparência legítima.
MÉDIA	backend/apps/relatorios/reports.py:29-33 backend/apps/relatorios/reports.py:243-251 backend/apps/relatorios/views.py:56-82	A-01 — _escopo() do motor de relatórios falha ABERTO e entrega dados de todos os clientes Um único usuário mal cadastrado enxerga e EXPORTA o parque, as medições, as anomalias, as ordens de serviço e o histórico de manutenção de todos os clientes da plataforma, em planilha pronta. Como o próprio módulo oferece download em CSV/XLSX/PDF, o vazamento é de base inteira, não de registro isolado. Reproduzido com prova de conceito. Condição: Exige uma conta de perfil CLIENTE_* com o campo cliente vazio. Não é um estado que o atacante produza sozinho, mas é alcançável: User.cliente é null=True (accounts/models.py:90-98) e o campo é editável por PATCH /api/usuarios/ (UserWriteSerializer, serializers.py:47-51), então basta um erro de cadastro ou a desvinculação de um cliente para a conta virar uma chave-mestra silenciosa.

Severidade	Arquivo:linha	Descrição
BAIXA	backend/apps/cadastros/ views.py:117-139 backend/apps/cadastros/ views.py:325-331 backend/apps/cadastros/models.py:735 backend/apps/coletas/ serializers.py:426-434	A-02 — Rota aceita equipamento de outro cliente (vetor do Portal fechado em 21/09) Com o Portal somente-consulta, o impacto residual é de integridade e não de confidencialidade: um analista interno pode, por engano, montar uma rota do Cliente A com um equipamento do Cliente B, contaminando a folha de campo, os relatórios e o indicador de cobertura de ativos. O risco relevante é de regressão — o código não impede o cenário grave, apenas deixou de oferecer a porta por onde se chegava a ele. Reproduzido com prova de conceito. Condição: O VETOR FOI FECHADO em 21/09/2026: RotaViewSet voltou a InternoEditaClienteVisualiza, e nenhum perfil de cliente escreve mais em /api/rotas/ — coberto por apps/osp/test_retorno_cliente.py::PortalSoC onsulta::test_nao_cria_rota. O que permanece é a LACUNA ESTRUTURAL: a guarda continua cega a relações ManyToMany, e CarregamentoSerializer.create continua copiando equipamentos sem conferir o dono. Hoje só a equipe interna alcança esse caminho, e ela tem escopo global por desenho — então não há quebra de inquilino, só risco de contaminação de dado por engano. Se a escrita do cliente voltar a ser liberada, como já foi em 18/09, o achado volta a ser ALTA sem nenhum aviso.
BAIXA	backend/apps/servicos/views.py:26-30 backend/apps/servicos/ atividades.py:96-97	A-04 — Duas cópias do filtro de tenant divergem do padrão fail-closed Nenhum impacto explorável no código atual. Dívida técnica de segurança: a quarta cópia da mesma regra, com semântica diferente das outras três, aumenta a chance de a próxima alteração repetir o erro de A-01.
2. Permissão definida no navegador		
BAIXA	frontend/src/lib/ permissions.ts:142-143 frontend/src/lib/permissions.ts:220 frontend/src/app/(admin)/ prestadores/ prestador-form.tsx:190-192 backend/apps/cadastros/ views.py:142-145 backend/apps/accounts/ permissions.py:55-66	A-05 — Tela de Prestadores é escondida por papel no front, mas /api/empresas/ é legível por qualquer conta Exposição limitada: Empresa é o cadastro da própria CONTRATADA (razão social, CNPJ, inscrição estadual, endereço, e-mail, telefone e nome do gestor), dados que já aparecem no timbre dos relatórios entregues. Não há dado de outro inquilino aqui. Vale corrigir porque o endpoint também alimenta o cabeçalho de laudos e pode crescer com campos sensíveis. Reproduzido com prova de conceito.
3. IDOR / controle de acesso		
CRÍTICA	backend/config/urls.py:16-17 backend/apps/accounts/views.py:26-29	A-06 — Endpoints JWT legados contornam MFA, bloqueio de conta e revogação de sessão Qualquer pessoa com um par e-mail/senha válido entra sem o segundo fator, sem limite de tentativas para adivinhar a senha e sem que o bloqueio automático de conta se aplique. Uma vez dentro, o token sobrevive ao logout, ao “encerrar outras sessões” e à própria redefinição de senha — e /api/auth/refresh/ o renova indefinidamente (ROTATE_REFRESH_TOKENS com 7 dias de vida). É a rota de persistência ideal depois de um vazamento de credencial. Reproduzido com prova de conceito.

Severidade	Arquivo:linha	Descrição
4. Chaves expostas		
ALTA	backend/config/settings.py:16-18 backend/config/settings.py:33 docker-compose.yml:18 docker-compose.yml:31 docker-compose.yml:34 backend/apps/core/management/commands/seed_demo.py:69-72 backend/apps/core/management/commands/seed_demo.py:88 backend/Dockerfile:25 backend/apps/accounts/security.py:151-153	A-07 — Segredos padrão embutidos no código, sem validação de startup que os recuse Com o SECRET_KEY conhecido um atacante forja tokens JWT de qualquer usuário, assina cookies de sessão, falsifica tokens CSRF, decifra os segredos TOTP de MFA guardados no banco e reverte os ip_hash do log de segurança. Com 'thermo123' presente, há um superusuário Django com senha pública; com DEBUG=True, qualquer erro devolve stack trace e settings. Condição: Os composes de produção (docker-compose.prod.yml:44-47 e docker-compose.http.yml:43-46) sobrescrevem o command e NÃO rodam seed_demo, e usam \${POSTGRES_PASSWORD} sem default — o caminho documentado de deploy está correto. O risco se concretiza quando alguém sobe a imagem fora desses composes, usa o docker-compose.yml de desenvolvimento num host alcançável, ou esquece SECRET_KEY no .env.prod.

5. Detalhamento técnico dos achados

Um bloco por achado, na ordem de severidade. Os blocos escuros reproduzem o código exatamente como está no repositório, com o número de linha original.

CRÍTICA

A-06 Endpoints JWT legados contornam MFA, bloqueio de conta e revogação de sessão

Categoria: 3. IDOR / controle de acesso

Arquivos: backend/config/urls.py:16-17 backend/apps/accounts/views.py:26-29

```
backend/config/urls.py
16: path("api/auth/login/", LoginView.as_view(), name="login"),
17: path("api/auth/refresh/", TokenRefreshView.as_view(), name="token_refresh"),

backend/apps/accounts/views.py
26: class LoginView(TokenObtainPairView):
29: serializer_class = LoginSerializer # só acrescenta o usuário ao payload
```

Por que é explorável

TokenObtainPairView do SimpleJWT declara `permission_classes = ()` e não define `throttle_classes`; como `REST_FRAMEWORK` não define `DEFAULT_THROTTLE_CLASSES` (`settings.py:135-140` define apenas as `RATES`), a rota fica pública e sem limite de tentativas. `LoginSerializer` (`accounts/serializers.py:70-78`) apenas chama `super().validate()` e anexa o usuário — não replica NENHUMA das travas do fluxo endurecido `LoginContextualView` (`auth_views.py:143-206`). O token emitido não carrega a claim 'sid', e `CookieJWTAuthentication.authenticate` (`authentication.py:26-28`) devolve `super().authenticate(request)` assim que há cabeçalho `Authorization`, saindo ANTES do `enforce_csrf` e da consulta a `SessaoAutenticacao` das linhas 45-52. O resultado é uma família de tokens invisível para todo o subsistema de sessões.

Proteções contornadas, uma a uma:

- MFA/TOTP obrigatório (`auth_views.py:178-194`)
- Bloqueio após 8 tentativas, campo `bloqueado_ate` (`auth_views.py:151-156` e `169-171`)
- Estado da conta `SUSPENDED/BLOCKED/PENDING` (`auth_serializers.py:61-62`)
- Separação portal x admin (`auth_views.py:173-176`)
- Throttle de 10 logins/minuto (`auth_views.py:145` + `throttles.py:4-5`)
- CSRF e a checagem de sessão viva (`authentication.py:45-52`)
- Logout, “encerrar outras sessões” e a revogação automática na troca/reset de senha (`auth_views.py:249-258`, `475-479` e `309`)

Prova de conceito executada

```
$ python manage.py test (saída real)

Teste executado contra a API (django test + APIClient):
HARDENED /api/auth/admin/login/ -> 428 (exige código MFA)
LEGACY /api/auth/login/ -> 200 ['access','refresh','user']
conta SUSPensa + BLOQUEADA: hardened -> 429 | legado -> 200
Bearer legado em /api/auth/me/ -> 200 alvo@thermoproactive.com
sessoes registradas no banco: 0
apos troca de senha + revogacao de TODAS as sessoes, Bearer legado -> 200
```

Impacto

Qualquer pessoa com um par e-mail/senha válido entra sem o segundo fator, sem limite de tentativas para adivinhar a senha e sem que o bloqueio automático de conta se aplique. Uma vez dentro, o token

sobrevive ao logout, ao “encerrar outras sessões” e à própria redefinição de senha — e `/api/auth/refresh/` o renova indefinidamente (ROTATE_REFRESH_TOKENS com 7 dias de vida). É a rota de persistência ideal depois de um vazamento de credencial.

Correção sugerida

Remover as duas linhas de `config/urls.py` e a classe `LoginView` de `accounts/views.py`: o frontend nunca as usa (`frontend/src/lib/api.ts:167` chama `/auth/{area}/login/` e `:56` chama `/auth/session/refresh/`). Se um token Bearer for necessário para integração máquina-a-máquina, criar um endpoint próprio que emita 'sid', grave `SessaoAutenticacao` e aplique MFA + throttle.

ALTA

A-03 /media/ é servido sem autenticação e expõe evidências técnicas e assinaturas digitais

Categoria: 1. Isolamento de inquilino

Arquivos: `deploy/nginx/app.conf.template:61-65` `deploy/nginx/app.http.conf:21-25` `backend/apps/coletas/models.py:614` `backend/apps/accounts/models.py:108-110` `backend/apps/cadastros/models.py:355` `backend/apps/cadastros/models.py:418` `backend/config/urls.py:33-34`

```
deploy/nginx/app.conf.template
61: location /media/ {
62: alias /var/www/media/;
63: access_log off;
64: expires 30d;
65: }

backend/apps/coletas/models.py
614: arquivo = models.ImageField("Arquivo", upload_to="achados/")

backend/apps/accounts/models.py
108: assinatura_digital = models.ImageField(
109: "Assinatura digital", upload_to="assinaturas/", null=True, blank=True
110: )
```

Por que é explorável

Todo o esforço de multi-tenant do projeto vive na camada DRF; os arquivos, porém, saem pelo Nginx direto do volume, sem passar pelo Django — não há `auth_request`, nem `X-Accel-Redirect`, nem token assinado na URL. O nome do arquivo é o nome original enviado pelo usuário (`FileSystemStorage` só acrescenta sufixo em caso de colisão), então os caminhos são curtos e enumeráveis: `/media/achados/IMG_0042.jpg`, `/media/assinaturas/joao.png`. O `access_log off` apaga até o rastro da tentativa. Vale nos dois `composes` de `deploy` e, em `DEBUG`, também no próprio Django (`config/urls.py:33-34`).

Impacto

Vazamento cross-tenant sem sequer precisar de conta: as fotos de termografia e vibração são a evidência técnica das anomalias de cada cliente (o que quebrou, onde, em que ativo), e as placas de identificação trazem fabricante, modelo e número de série do parque alheio. Pior, `/media/assinaturas/` guarda a imagem da assinatura do responsável técnico usada para assinar laudos — baixá-la permite forjar um laudo com aparência legítima.

Correção sugerida

Servir mídia por uma view Django autenticada que reaplique `escopo_cliente` sobre o dono do arquivo e delegue a entrega ao Nginx via `X-Accel-Redirect` (`location internal`), ou usar `auth_request` apontando para um endpoint de autorização. Em paralelo, trocar o nome do arquivo por UUID (`upload_to` com função) para eliminar a enumeração, e reativar o `access_log`.

ALTA

A-07 Segredos padrão embutidos no código, sem validação de startup que os recuse

Categoria: 4. Chaves expostas

Arquivos: backend/config/settings.py:16-18 backend/config/settings.py:33 docker-compose.yml:18 docker-compose.yml:31 docker-compose.yml:34 backend/apps/core/management/commands/seed_demo.py:69-72 backend/apps/core/management/commands/seed_demo.py:88 backend/Dockerfile:25 backend/apps/accounts/security.py:151-153

```
backend/config/settings.py
16: DEBUG=(bool, True),
17: SECRET_KEY=(str, "dev-insecure-change-me-em-producao"),
18: ALLOWED_HOSTS=(list, ["*"]),
33: SECRET_KEY = env("SECRET_KEY") # nunca levanta erro: o default cobre a ausência

docker-compose.yml
18: POSTGRES_PASSWORD: thermo
31: SECRET_KEY: docker-dev-secret-troque-em-producao
34: DATABASE_URL: postgres://thermo:thermo@db:5432/thermoproactive

backend/apps/core/management/commands/seed_demo.py
69: "admin@thermoproactive.com": (
71: {"is_staff": True, "is_superuser": True, "nivel": "MASTER"},
88: user.set_password("thermo123")

backend/Dockerfile
25: CMD [... "&& { python manage.py seed_demo || echo 'seed_demo ignorado'; } && exec gunicorn ..."]
```

Por que é explorável

django-environ só usa o default quando a variável está AUSENTE — e é exatamente esse o caso perigoso: subir em produção sem SECRET_KEY não quebra nada, o Django simplesmente assume a string pública do repositório. Não há nenhum django.core.checks nem assert de boot que recuse o valor conhecido, e o mesmo vale para DEBUG=True e ALLOWED_HOSTS=['*'] como defaults. O impacto é amplificado porque accounts/security.py:151-153 deriva a chave Fernet que cifra os segredos TOTP do MFA a partir do SECRET_KEY, e security.py:30-31 usa o mesmo valor como chave HMAC dos ip_hash. Por fim, o CMD padrão da imagem backend roda seed_demo, que cria admin@thermoproactive.com com is_superuser=True e senha 'thermo123'.

Condições de explorabilidade

Os composes de produção (docker-compose.prod.yml:44-47 e docker-compose.http.yml:43-46) sobrescrevem o command e NÃO rodam seed_demo, e usam \${POSTGRES_PASSWORD} sem default — o caminho documentado de deploy está correto. O risco se concretiza quando alguém sobe a imagem fora desses composes, usa o docker-compose.yml de desenvolvimento num host alcançável, ou esquece SECRET_KEY no .env.prod.

Impacto

Com o SECRET_KEY conhecido um atacante forja tokens JWT de qualquer usuário, assina cookies de sessão, falsifica tokens CSRF, decifra os segredos TOTP de MFA guardados no banco e reverte os ip_hash do log de segurança. Com 'thermo123' presente, há um superusuário Django com senha pública; com DEBUG=True, qualquer erro devolve stack trace e settings.

Correção sugerida

1) Remover o default de SECRET_KEY (env('SECRET_KEY') sem default levanta ImproperlyConfigured) ou adicionar um django.core.checks que falhe o boot quando DEBUG=False e o valor for um dos conhecidos. 2) DEBUG default False e ALLOWED_HOSTS sem '*'. 3) Trocar os valores literais do docker-compose.yml por \${VAR:?erro} lendo um .env local. 4) Fazer seed_demo recusar rodar com

DEBUG=False e sortear a senha, imprimindo-a no log. 5) Derivar a chave Fernet do MFA de uma variável própria (MFA_ENCRYPTION_KEY), desacoplando-a do SECRET_KEY.

MÉDIA

A-01 _escopo() do motor de relatórios falha ABERTO e entrega dados de todos os clientes

Categoria: 1. Isolamento de inquilino

Arquivos: backend/apps/relatorios/reports.py:29-33 backend/apps/relatorios/reports.py:243-251 backend/apps/relatorios/views.py:56-82

```
backend/apps/relatorios/reports.py
29: def _escopo(qs, user, campo_cliente):
30:     """Perfis cliente: restringe ao próprio cliente."""
31:     if user.is_cliente and user.cliente_id:
32:         return qs.filter(**{campo_cliente: user.cliente_id})
33:     return qs # <- FAIL-OPEN: cliente sem vínculo recebe TUDO

comparar com backend/apps/coletas/views.py:49-53 (fail-closed, correto)
49: if user.is_cliente:
50:     if not user.cliente_id:
51:         return qs.none()
52:     return qs.filter(**{campo_cliente: user.cliente_id})
```

Por que é explorável

O projeto tem três implementações do mesmo filtro de tenant e duas delas documentam explicitamente a falha fechada (“Falha fechada: cliente sem cliente_id não vê nada, em vez de cair no qs inteiro sem filtro”). A do módulo relatorios não seguiu a regra: o “and user.cliente_id” transforma a ausência de vínculo em ausência de filtro. Atinge os cinco relatórios liberados ao cliente (técnico, gerencial, equipamento, falhas, histórico — reports.py:244-250), em JSON, CSV, XLSX e PDF, via GET /api/relatorios/<key>/?formato=...

Prova de conceito executada

```
$ python manage.py test (saída real)

Teste executado contra a API (django test + APIClient), usuario CLIENTE_PCM sem vínculo:
cliente_id do usuario: None
_escopo(reports.py) devolveu 1 clientes -> ['Cliente B (vitima)']
GET /api/relatorios/gerencial/ -> 200 linhas: [['Cliente B (vitima)', 0, 0, 0, '0%', 0]]
```

Condições de explorabilidade

Exige uma conta de perfil CLIENTE_* com o campo cliente vazio. Não é um estado que o atacante produza sozinho, mas é alcançável: User.cliente é null=True (accounts/models.py:90-98) e o campo é editável por PATCH /api/usuarios/ (UserWriteSerializer, serializers.py:47-51), então basta um erro de cadastro ou a desvinculação de um cliente para a conta virar uma chave-mestra silenciosa.

Impacto

Um único usuário mal cadastrado enxerga e EXPORTA o parque, as medições, as anomalias, as ordens de serviço e o histórico de manutenção de todos os clientes da plataforma, em planilha pronta. Como o próprio módulo oferece download em CSV/XLSX/PDF, o vazamento é de base inteira, não de registro isolado.

Correção sugerida

Fazer _escopo devolver qs.none() quando user.is_cliente e não há cliente_id, alinhando com coletas/views.py:49-53. Melhor ainda: extrair as três cópias para uma única função em apps/core e importá-la nos três módulos, para a regra não poder divergir de novo.

BAIXA

A-02 Rota aceita equipamento de outro cliente (vetor do Portal fechado em 21/09)**Categoria: 1. Isolamento de inquilino**

Arquivos: backend/apps/cadastros/views.py:117-139 backend/apps/cadastros/views.py:325-331 backend/apps/cadastros/models.py:735 backend/apps/coletas/serializers.py:426-434

```
backend/apps/cadastros/views.py (_impedir_apontar_para_outro_cliente)
129: primeiro_campo, _, resto = self.campo_cliente.partition("__")
130: if primeiro_campo not in validated_data:
131: return # <- só o 1o campo do caminho é validado

317: class RotaViewSet(BaseCadastroViewSet):
318: campo_cliente = "cliente" # <- o M2M equipamentos fica de fora
(a permissão herdada passou a ser InternoEditaClienteVisualiza em 21/09)

backend/apps/coletas/serializers.py (CarregamentoSerializer.create)
428: equipamentos = carregamento.rota.equipamentos.order_by(...)
431: ItemInspecao.objects.bulk_create([
432: ItemInspecao(carregamento=carregamento, equipamento=eq, ordem=i)
433: for i, eq in enumerate(equipamentos, start=1)
434: ]) # <- copia TODOS, sem checar de quem são
```

Por que é explorável

A guarda de escrita valida apenas `campo_cliente.partition('__')[0]`, isto é, o campo 'cliente' da própria Rota. O ManyToMany 'equipamentos' (cadastros/models.py:735) nunca é conferido. Quando o Portal escrevia em `/api/rotas/`, isso era uma quebra direta de inquilino: o Master de um cliente gravava IDs de equipamentos de qualquer outro, e o vazamento se materializava no carregamento da rota — `CarregamentoSerializer.create` copia todos os equipamentos para `ItemInspecao` sem checar o dono, e o Carregamento resultante pertence a quem montou a rota, então `/api/itens-inspecao/` devolvia o item alheio como se fosse dele.

Prova de conceito executada

```
$ python manage.py test (saída real)
PoC de 20/09, quando o Portal ainda escrevia em /api/rotas/:
POST /api/rotas/ com equipamento do cliente B -> 201
interno carrega a rota -> 201
Master do cliente A le /api/itens-inspecao/ -> 200
VAZOU: TAG-SECRETA-B-001 | Turbina confidencial da vitima |
Area secreta B / Setor secreto B

Mesmo POST apos a mudanca de 21/09 (apps/osp/test_retorno_cliente.py):
POST /api/rotas/ como Master do cliente -> 403
```

Condições de explorabilidade

O VETOR FOI FECHADO em 21/09/2026: `RotaViewSet` voltou a `InternoEditaClienteVisualiza`, e nenhum perfil de cliente escreve mais em `/api/rotas/` — coberto por `apps/osp/test_retorno_cliente.py::PortalSoConsulta::test_nao_cria_rota`. O que permanece é a LACUNA ESTRUTURAL: a guarda continua cega a relações ManyToMany, e `CarregamentoSerializer.create` continua copiando equipamentos sem conferir o dono. Hoje só a equipe interna alcança esse caminho, e ela tem escopo global por desenho — então não há quebra de inquilino, só risco de contaminação de dado por engano. Se a escrita do cliente voltar a ser liberada, como já foi em 18/09, o achado volta a ser ALTA sem nenhum aviso.

Impacto

Com o Portal somente-consulta, o impacto residual é de integridade e não de confidencialidade: um analista interno pode, por engano, montar uma rota do Cliente A com um equipamento do Cliente B, contaminando a folha de campo, os relatórios e o indicador de cobertura de ativos. O risco relevante é de regressão — o código não impede o cenário grave, apenas deixou de oferecer a porta por onde se chegava a ele.

Correção sugerida

Validar o M2M na escrita, agora pelo dono da ROTA e não pelo usuário: em `RotaSerializer.validate`, recusar qualquer equipamento cujo `setor__area__cliente_id` seja diferente do cliente da rota — vale para interno e para cliente, e é exatamente a checagem que `servicos/atividades.py:52-53` já faz no fluxo corretivo. Em defesa em profundidade, replicar o filtro em `CarregamentoSerializer.create`.

BAIXA

A-04 Duas cópias do filtro de tenant divergem do padrão fail-closed

Categoria: 1. Isolamento de inquilino

Arquivos: `backend/apps/servicos/views.py:26-30` `backend/apps/servicos/atividades.py:96-97`

```
backend/apps/servicos/views.py
26: def escopo_cliente(qs, user, campo_cliente="cliente"):
27:     """Perfis cliente só enxergam os próprios serviços (Portal - item 2.7)."""
28:     if user.is_cliente:
29:         return qs.filter(**{campo_cliente: user.cliente_id}) # sem guarda de None
30:     return qs

backend/apps/servicos/atividades.py
96: if self.request.user.is_cliente:
97:     qs = qs.filter(cliente_id=self.request.user.cliente_id)
```

Por que é explorável

Ao contrário de A-01, aqui NÃO há vazamento hoje: `cliente_id` `None` vira “`cliente_id IS NULL`” e `ServicoCampo.cliente` (`servicos/models.py:48`) e `Carregamento.cliente` são NOT NULL, então o resultado é vazio por acidente, não por desenho. O risco é de manutenção: qualquer futura coluna de tenant que aceite NULL, ou a reutilização deste helper com outro `campo_cliente`, transforma o acidente em vazamento.

Impacto

Nenhum impacto explorável no código atual. Dívida técnica de segurança: a quarta cópia da mesma regra, com semântica diferente das outras três, aumenta a chance de a próxima alteração repetir o erro de A-01.

Correção sugerida

Importar o helper único (ver A-01) em `apps/servicos/views.py` e `apps/servicos/atividades.py` e apagar as cópias locais.

BAIXA

A-05 Tela de Prestadores é escondida por papel no front, mas /api/empresas/ é legível por qualquer conta**Categoria: 2. Permissão definida no navegador**

Arquivos: frontend/src/lib/permissions.ts:142-143 frontend/src/lib/permissions.ts:220 frontend/src/app/(admin)/prestadores/prestador-form.tsx:190-192 backend/apps/cadastros/views.py:142-145 backend/apps/accounts/permissions.py:55-66

```
frontend/src/lib/permissions.ts
142: case "prestadores:gerenciar":
143: return interno;
220: { prefixo: "/prestadores", exige: "prestadores:gerenciar" },

frontend/src/app/(admin)/prestadores/prestador-form.tsx
190: await api(`/empresas/${prestadorId}`, { method: "PATCH", body });

backend/apps/cadastros/views.py
142: class EmpresaViewSet(BaseCadastroViewSet): # herda InternoEditaClienteVisualiza
143: queryset = Empresa.objects.ativos() # campo_cliente = None (sem escopo)

backend/apps/accounts/permissions.py (InternoEditaClienteVisualiza)
59: if request.method in SAFE_METHODS:
60: return True # <- qualquer autenticado LÊ
```

Por que é explorável

A rota /prestadores só aparece para usuário interno, mas ela é apenas um rótulo: a API por trás é /api/empresas/, cujo ViewSet herda a permissão padrão de BaseCadastroViewSet, que libera leitura a qualquer autenticado, e não declara campo_cliente. A escrita está corretamente bloqueada (PATCH/POST/DELETE exigem perfil interno).

Prova de conceito executada

```
$ python manage.py test (saída real)
Teste executado contra a API, usuario CLIENTE_MANUT / JUNIOR do Portal:
GET /api/empresas/ -> 200
{'nome': 'ThermoProActive Servicos Ltda', 'cnpj': '39.923.567/0001-75',
'email': 'financeiro@thermoproactive.com', 'telefone': '(19) 99999-0000',
'contato_gestor': 'Diretor Tecnico'}
POST /api/empresas/ (escrita) -> 403 (corretamente bloqueado)
```

Impacto

Exposição limitada: Empresa é o cadastro da própria CONTRATADA (razão social, CNPJ, inscrição estadual, endereço, e-mail, telefone e nome do gestor), dados que já aparecem no timbre dos relatórios entregues. Não há dado de outro inquilino aqui. Vale corrigir porque o endpoint também alimenta o cabeçalho de laudos e pode crescer com campos sensíveis.

Correção sugerida

Trocar a permission_class de EmpresaViewSet para IsInterno (leitura e escrita), alinhando com o gate 'prestadores:gerenciar' do front. Se o Portal precisar do nome/logo do prestador para o cabeçalho, expor um endpoint público mínimo com apenas esses campos.

6. Recomendações priorizadas

P1 = antes do próximo deploy. P2 = nesta sprint. P3 = backlog de higiene, sem exploração conhecida hoje. ✓ = já corrigido depois da primeira emissão deste relatório.

Prio.	Ação	Achado	Como
P1	Remover /api/auth/login/ e /api/auth/refresh/ de config/urls.py	A-06	Fecha o único achado crítico. O frontend não usa essas rotas; é uma remoção de 2 linhas. Depois da remoção, revogar as sessões existentes e forçar novo login.
P1	Colocar /media/ atrás de autenticação e escopo de cliente	A-03	View Django + X-Accel-Redirect (ou auth_request no Nginx), marcando a location como internal. Trocar os nomes de arquivo por UUID e reativar o access_log.
P2	Remover os defaults de SECRET_KEY/DEBUG/ALLOWED_HOSTS e adicionar checagem de boot	A-07	Um django.core.checks que falhe com DEBUG=False + segredo conhecido. Tirar os literais do docker-compose.yml, travar seed_demo fora de DEBUG e rotacionar a chave de produção.
P2	Unificar o filtro de tenant num único helper fail-closed em apps/core	A-01, A-04	Eliminar as quatro cópias. Corrigir o fail-open de reports.py:29-33 no mesmo passo e cobrir com teste o caso do usuário cliente sem vínculo.
P3	Validar o M2M de Rota.equipamentos na escrita	A-02	Deixou de ser P1 em 21/09, quando o Portal voltou a ser somente-consulta e o vetor sumiu. Recusar equipamento cujo dono não seja o cliente DA ROTA (vale para interno também) em RotaSerializer.validate e, em defesa em profundidade, filtrar por cliente em CarregamentoSerializer.create.
P3	Restringir /api/empresas/ a usuários internos	A-05	Trocar a permission_class para IsInterno, alinhando o backend ao gate do front.
P3	Adicionar um teste de fumaça de autorização ao pipeline	–	Um teste parametrizado que, para cada rota registrada nos routers, autentique como cliente de outro tenant e exija 403/404. Transforma o padrão “lembre de declarar campo_cliente” numa garantia verificada.
✓	Reconciliar a matriz de permissões do frontend com a do backend	–	FEITO em 21/09: a divergência (osp:criar, osp:mudar-status e operacional:criar mais restritos no front do que no backend) deixou de existir quando o Portal voltou a ser somente-consulta. frontend/src/lib/permissions.ts:8-30 registra a decisão e a habilidade nova osp:responder.

7. Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar. Cada issue está delimitada entre --- ISSUE n --- e --- FIM ISSUE n ---. Achados triviais do mesmo tema foram agrupados numa issue só, para não gerar spam no repositório.

--- ISSUE 1 ---

```
# [Segurança] Endpoints JWT legados /api/auth/login/ e /api/auth/refresh/ contornam MFA, bloqueio de conta e revogação de sessão

**Labels sugeridas:** `security`, `severidade:crítica`

**Achado(s) da auditoria:** A-06

---

**Problema**

TokenObtainPairView do SimpleJWT declara permission_classes = () e não define throttle_classes; como REST_FRAMEWORK não define DEFAULT_THROTTLE_CLASSES (settings.py:135-140 define apenas as RATES), a rota fica pública e sem limite de tentativas. LoginSerializer (accounts/serializers.py:70-78) apenas chama super().validate() e anexa o usuário – não replica NENHUMA das travas do fluxo endurecido LoginContextualView (auth_views.py:143-206). O token emitido não carrega a claim 'sid', e CookieJWTAuthentication.authenticate (authentication.py:26-28) devolve super().authenticate(request) assim que há cabeçalho Authorization, saindo ANTES do enforce_csrf e da consulta a SessaoAutenticacao das linhas 45-52. O resultado é uma família de tokens invisível para todo o subsistema de sessões.

O endpoint contorna, uma a uma, as seguintes proteções:

- MFA/TOTP obrigatório (auth_views.py:178-194)
- Bloqueio após 8 tentativas, campo bloqueado_ate (auth_views.py:151-156 e 169-171)
- Estado da conta SUSPENDED/BLOCKED/PENDING (auth_serializers.py:61-62)
- Separação portal × admin (auth_views.py:173-176)
- Throttle de 10 logins/minuto (auth_views.py:145 + throttles.py:4-5)
- CSRF e a checagem de sessão viva (authentication.py:45-52)
- Logout, “encerrar outras sessões” e a revogação automática na troca/reset de senha (auth_views.py:249-258, 475-479 e 309)

**Evidência**

Arquivos: `backend/config/urls.py:16-17`, `backend/apps/accounts/views.py:26-29`

```python
backend/config/urls.py
16: path("api/auth/login/", LoginView.as_view(), name="login"),
17: path("api/auth/refresh/", TokenRefreshView.as_view(), name="token_refresh"),

backend/apps/accounts/views.py
26: class LoginView(TokenObtainPairView):
29: serializer_class = LoginSerializer # só acrescenta o usuário ao payload
```

Prova de conceito reproduzida (`manage.py test` com `APIClient`):

```text
Teste executado contra a API (django test + APIClient):
HARDENED /api/auth/admin/login/ -> 428 (exige código MFA)
LEGACY /api/auth/login/ -> 200 ['access','refresh','user']
conta SUSPENSA + BLOQUEADA: hardened -> 429 | legado -> 200
Bearer legado em /api/auth/me/ -> 200 alvo@thermoproactive.com
sessoes registradas no banco: 0
apos troca de senha + revogacao de TODAS as sessoes, Bearer legado -> 200
```

**Impacto**

Qualquer pessoa com um par e-mail/senha válido entra sem o segundo fator, sem limite de tentativas para adivinhar a senha e sem que o bloqueio automático de conta se aplique. Uma vez dentro, o token sobrevive ao logout, ao “encerrar outras sessões” e à própria redefinição de senha – e /api/auth/refresh/ o renova indefinidamente (ROTATE_REFRESH_TOKENS com 7 dias de vida). É a rota de persistência ideal depois de um vazamento de credencial.
```

****Sugestão de correção****

Remover as duas linhas de config/urls.py e a classe LoginView de accounts/views.py: o frontend nunca as usa (frontend/src/lib/api.ts:167 chama /auth/{area}/login/ e :56 chama /auth/session/refresh/). Se um token Bearer for necessário para integração máquina-a-máquina, criar um endpoint próprio que emita 'sid', grave SessaoAutenticacao e aplique MFA + throttle.

****Critérios de aceite****

- [] path('api/auth/login/') e path('api/auth/refresh/') removidos de backend/config/urls.py
- [] Classe LoginView e LoginSerializer removidas (ou reescritas com sid + MFA + throttle)
- [] Teste automatizado: POST /api/auth/login/ devolve 404
- [] Teste automatizado: conta com MFA ativo não obtém token por nenhuma rota sem o código TOTP
- [] Teste automatizado: token obtido antes de _revogar_todas() devolve 401 depois dela
- [] Sessões existentes revogadas e novo login forçado após o deploy da correção

--- FIM ISSUE 1 ---**--- ISSUE 2 ---**

[Segurança] /media/ é servido sem autenticação e expõe evidências técnicas e assinaturas digitais

****Labels sugeridas:**** `security`, `severidade:alta`

****Achado(s) da auditoria:**** A-03

****Problema****

Todo o esforço de multi-tenant do projeto vive na camada DRF; os arquivos, porém, saem pelo Nginx direto do volume, sem passar pelo Django — não há auth_request, nem X-Accel-Redirect, nem token assinado na URL. O nome do arquivo é o nome original enviado pelo usuário (FileSystemStorage só acrescenta sufixo em caso de colisão), então os caminhos são curtos e enumeráveis: /media/achados/IMG_0042.jpg, /media/assinaturas/joao.png. O access_log off apaga até o rastro da tentativa. Vale nos dois composes de deploy e, em DEBUG, também no próprio Django (config/urls.py:33-34).

****Evidência****

Arquivos: `deploy/nginx/app.conf.template:61-65`, `deploy/nginx/app.http.conf:21-25`,
 `backend/apps/coletas/models.py:614`, `backend/apps/accounts/models.py:108-110`,
 `backend/apps/cadastros/models.py:355`, `backend/apps/cadastros/models.py:418`, `backend/config/urls.py:33-34`

```python

```
deploy/nginx/app.conf.template
61: location /media/ {
62: alias /var/www/media/;
63: access_log off;
64: expires 30d;
65: }
```

```
backend/apps/coletas/models.py
614: arquivo = models.ImageField("Arquivo", upload_to="achados/")
```

```
backend/apps/accounts/models.py
108: assinatura_digital = models.ImageField(
109: "Assinatura digital", upload_to="assinaturas/", null=True, blank=True
110:)
```
```

****Impacto****

Vazamento cross-tenant sem sequer precisar de conta: as fotos de termografia e vibração são a evidência técnica das anomalias de cada cliente (o que quebrou, onde, em que ativo), e as placas de identificação trazem fabricante, modelo e número de série do parque alheio. Pior, /media/assinaturas/ guarda a imagem da assinatura do responsável técnico usada para assinar laudos — baixá-la permite forjar um laudo com aparência legítima.

****Sugestão de correção****

Servir mídia por uma view Django autenticada que reaplique escopo_cliente sobre o dono do arquivo e delegue a entrega ao Nginx via X-Accel-Redirect (location internal), ou usar auth_request apontando para um endpoint de autorização. Em paralelo, trocar o nome do arquivo por UUID (upload_to com função) para eliminar a enumeração, e reativar o access_log.

****Critérios de aceite****

- [] GET /media/achados/<arquivo> sem cookie de sessão devolve 401/403
- [] GET /media/achados/<arquivo> autenticado como cliente de OUTRO tenant devolve 403/404
- [] location /media/ marcada como 'internal' no Nginx (ou protegida por auth_request)
- [] upload_to de achados, assinaturas e placas gera nome aleatório (UUID)
- [] access_log reativado para /media/

--- FIM ISSUE 2 ---**--- ISSUE 3 ---**

[Segurança] Rota aceita equipamento de outro cliente e vaza o parque alheio na folha de campo

****Labels sugeridas:**** `security`, `severidade:alta`

****Achado(s) da auditoria:**** A-02

****Problema****

A guarda de escrita valida apenas `campo_cliente.partition('__')[0]`, isto é, o campo 'cliente' da própria Rota. O `ManyToMany` 'equipamentos' (`cadastros/models.py:735`) nunca é conferido. Quando o Portal escrevia em `/api/rotas/`, isso era uma quebra direta de inquilino: o Master de um cliente gravava IDs de equipamentos de qualquer outro, e o vazamento se materializava no carregamento da rota – `CarregamentoSerializer.create` copia todos os equipamentos para `ItemInspecao` sem checar o dono, e o `Carregamento` resultante pertence a quem montou a rota, então `/api/itens-inspecao/` devolvia o item alheio como se fosse dele.

****Evidência****

Arquivos: ``backend/apps/cadastros/views.py:117-139``, ``backend/apps/cadastros/views.py:325-331``,
``backend/apps/cadastros/models.py:735``, ``backend/apps/coletas/serializers.py:426-434``

```
```python
backend/apps/cadastros/views.py (_impedir_apontar_para_outro_cliente)
129: primeiro_campo, _, resto = self.campo_cliente.partition("__")
130: if primeiro_campo not in validated_data:
131: return # <- só o 1o campo do caminho é validado

317: class RotaViewSet(BaseCadastroViewSet):
318: campo_cliente = "cliente" # <- o M2M equipamentos fica de fora
(a permissão herdada passou a ser InternoEditaClienteVisualiza em 21/09)

backend/apps/coletas/serializers.py (CarregamentoSerializer.create)
428: equipamentos = carregamento.rota.equipamentos.order_by(...)
431: ItemInspecao.objects.bulk_create([
432: ItemInspecao(carregamento=carregamento, equipamento=eq, ordem=i)
433: for i, eq in enumerate(equipamentos, start=1)
434:]) # <- copia TODOS, sem checar de quem são
```
```

Prova de conceito reproduzida (``manage.py test`` com ``APIClient``):

```
```text
PoC de 20/09, quando o Portal ainda escrevia em /api/rotas/:
POST /api/rotas/ com equipamento do cliente B -> 201
interno carrega a rota -> 201
Master do cliente A le /api/itens-inspecao/ -> 200
```

VAZOU: TAG-SECRETA-B-001 | Turbina confidencial da vítima |  
Area secreta B / Setor secreto B

Mesmo POST apos a mudanca de 21/09 (apps/osp/test\_retorno\_cliente.py):  
POST /api/rotas/ como Master do cliente -> 403  
``

#### **\*\*Condições de explorabilidade\*\***

O VETOR FOI FECHADO em 21/09/2026: RotaViewSet voltou a InternoEditaClienteVisualiza, e nenhum perfil de cliente escreve mais em /api/rotas/ – coberto por apps/osp/test\_retorno\_cliente.py:PortalSoConsulta::test\_ao\_cria\_rota. O que permanece é a LACUNA ESTRUTURAL: a guarda continua cega a relações ManyToMany, e CarregamentoSerializer.create continua copiando equipamentos sem conferir o dono. Hoje só a equipe interna alcança esse caminho, e ela tem escopo global por desenho – então não há quebra de inquilino, só risco de contaminação de dado por engano. Se a escrita do cliente voltar a ser liberada, como já foi em 18/09, o achado volta a ser ALTA sem nenhum aviso.

#### **\*\*Impacto\*\***

Com o Portal somente-consulta, o impacto residual é de integridade e não de confidencialidade: um analista interno pode, por engano, montar uma rota do Cliente A com um equipamento do Cliente B, contaminando a folha de campo, os relatórios e o indicador de cobertura de ativos. O risco relevante é de regressão – o código não impede o cenário grave, apenas deixou de oferecer a porta por onde se chegava a ele.

#### **\*\*Sugestão de correção\*\***

Validar o M2M na escrita, agora pelo dono da ROTA e não pelo usuário: em RotaSerializer.validate, recusar qualquer equipamento cujo setor\_\_area\_\_cliente\_id seja diferente do cliente da rota – vale para interno e para cliente, e é exatamente a checagem que servicos/atividades.py:52-53 já faz no fluxo corretivo. Em defesa em profundidade, replicar o filtro em CarregamentoSerializer.create.

#### **\*\*Critérios de aceite\*\***

- [ ] PATCH/POST /api/rotas/ com equipamento de outro cliente devolve 400, inclusive para interno
- [ ] CarregamentoSerializer.create ignora (ou recusa) equipamento fora do cliente do carregamento
- [ ] Teste de regressão em apps/cadastros/test\_escopo\_cliente.py cobrindo o M2M de Rota
- [ ] Auditoria única dos demais M2M do projeto em busca do mesmo padrão

### **--- FIM ISSUE 3 ---**

### **--- ISSUE 4 ---**

# [Segurança] Segredos padrão embutidos no código, sem validação de startup que os recuse

**\*\*Labels sugeridas:\*\*** `security`, `severidade:alta`

**\*\*Achado(s) da auditoria:\*\*** A-07

---

#### **\*\*Problema\*\***

django-environ só usa o default quando a variável está AUSENTE – e é exatamente esse o caso perigoso: subir em produção sem SECRET\_KEY não quebra nada, o Django simplesmente assume a string pública do repositório. Não há nenhum django.core.checks nem assert de boot que recuse o valor conhecido, e o mesmo vale para DEBUG=True e ALLOWED\_HOSTS=['\*'] como defaults. O impacto é amplificado porque accounts/security.py:151-153 deriva a chave Fernet que cifra os segredos TOTP do MFA a partir do SECRET\_KEY, e security.py:30-31 usa o mesmo valor como chave HMAC dos ip\_hash. Por fim, o CMD padrão da imagem backend roda seed\_demo, que cria admin@thermoproactive.com com is\_superuser=True e senha 'thermo123'.

#### **\*\*Evidência\*\***

Arquivos: `backend/config/settings.py:16-18`, `backend/config/settings.py:33`, `docker-compose.yml:18`, `docker-compose.yml:31`, `docker-compose.yml:34`, `backend/apps/core/management/commands/seed\_demo.py:69-72`, `backend/apps/core/management/commands/seed\_demo.py:88`, `backend/Dockerfile:25`, `backend/apps/accounts/security.py:151-153`

``python

```
backend/config/settings.py
16: DEBUG=(bool, True),
17: SECRET_KEY=(str, "dev-insecure-change-me-em-producao"),
18: ALLOWED_HOSTS=(list, ["*"]),
33: SECRET_KEY = env("SECRET_KEY") # nunca levanta erro: o default cobre a ausência
```

```

docker-compose.yml
18: POSTGRES_PASSWORD: thermo
31: SECRET_KEY: docker-dev-secret-troque-em-producao
34: DATABASE_URL: postgres://thermo:thermo@db:5432/thermoproactive

backend/apps/core/management/commands/seed_demo.py
69: "admin@thermoproactive.com": (
71: {"is_staff": True, "is_superuser": True, "nivel": "MASTER"},
88: user.set_password("thermo123")

backend/Dockerfile
25: CMD [... "&& { python manage.py seed_demo || echo 'seed_demo ignorado'; } && exec gunicorn ..."]
```



**Condições de explorabilidade**



Os composes de produção (docker-compose.prod.yml:44-47 e docker-compose.http.yml:43-46) sobrescrevem o command e NÃO rodam seed_demo, e usam ${POSTGRES_PASSWORD} sem default – o caminho documentado de deploy está correto. O risco se concretiza quando alguém sobe a imagem fora desses composes, usa o docker-compose.yml de desenvolvimento num host alcançável, ou esquece SECRET_KEY no .env.prod.



**Impacto**



Com o SECRET_KEY conhecido um atacante forja tokens JWT de qualquer usuário, assina cookies de sessão, falsifica tokens CSRF, decifra os segredos TOTP de MFA guardados no banco e reverte os ip_hash do log de segurança. Com 'thermo123' presente, há um superusuário Django com senha pública; com DEBUG=True, qualquer erro devolve stack trace e settings.



**Sugestão de correção**



1) Remover o default de SECRET_KEY (env('SECRET_KEY') sem default levanta ImproperlyConfigured) ou adicionar um django.core.checks que falhe o boot quando DEBUG=False e o valor for um dos conhecidos. 2) DEBUG default False e ALLOWED_HOSTS sem '*'. 3) Trocar os valores literais do docker-compose.yml por ${VAR:erro} lendo um .env local. 4) Fazer seed_demo recusar rodar com DEBUG=False e sortear a senha, imprimindo-a no log. 5) Derivar a chave Fernet do MFA de uma variável própria (MFA_ENCRYPTION_KEY), desacoplando-a do SECRET_KEY.



**Critérios de aceite**



- [ ] Boot com DEBUG=False e SECRET_KEY ausente/default falha com mensagem clara
- [ ] settings.py: DEBUG default False e ALLOWED_HOSTS sem '*'
- [ ] docker-compose.yml sem senha ou chave literal (usa ${VAR:?} + .env)
- [ ] seed_demo recusa rodar com DEBUG=False e não usa senha fixa
- [ ] Segredo do MFA derivado de variável própria, não do SECRET_KEY
- [ ] Chave de produção rotacionada depois da correção (a antiga está no repositório)

```

--- FIM ISSUE 4 ---**--- ISSUE 5 ---**

```

# [Segurança] _escopo() do motor de relatórios falha aberto e entrega dados de todos os clientes

**Labels sugeridas:** `security`, `severidade:média`

**Achado(s) da auditoria:** A-01

---

**Problema**

O projeto tem três implementações do mesmo filtro de tenant e duas delas documentam explicitamente a falha fechada ("Falha fechada: cliente sem cliente_id não vê nada, em vez de cair no qs inteiro sem filtro"). A do módulo relatorios não seguiu a regra: o "and user.cliente_id" transforma a ausência de vínculo em ausência de filtro. Atinge os cinco relatórios liberados ao cliente (técnico, gerencial, equipamento, falhas, histórico – reports.py:244-250), em JSON, CSV, XLSX e PDF, via GET /api/relatorios/<key>/?formato=...

**Evidência**

Arquivos: `backend/apps/relatorios/reports.py:29-33`, `backend/apps/relatorios/reports.py:243-251`,
`backend/apps/relatorios/views.py:56-82`

```python
backend/apps/relatorios/reports.py

```

```

29: def _escopo(qs, user, campo_cliente):
30: """Perfis cliente: restringe ao próprio cliente."""
31: if user.is_cliente and user.cliente_id:
32: return qs.filter(**{campo_cliente: user.cliente_id})
33: return qs # <- FAIL-OPEN: cliente sem vínculo recebe TUDO

comparar com backend/apps/coletas/views.py:49-53 (fail-closed, correto)
49: if user.is_cliente:
50: if not user.cliente_id:
51: return qs.none()
52: return qs.filter(**{campo_cliente: user.cliente_id})
```

Prova de conceito reproduzida (`manage.py test` com `APIClient`):

```text
Teste executado contra a API (django test + APIClient), usuario CLIENTE_PCM sem vinculo:
cliente_id do usuario: None
_escopo(reports.py) devolveu 1 clientes -> ['Cliente B (vitima)']
GET /api/relatorios/gerencial/ -> 200 linhas: [['Cliente B (vitima)', 0, 0, 0, '0%', 0]]
```

**Condições de explorabilidade**

Exige uma conta de perfil CLIENTE_* com o campo cliente vazio. Não é um estado que o atacante produza sozinho, mas é alcançável: User.cliente é null=True (accounts/models.py:90-98) e o campo é editável por PATCH /api/usuarios/ (UserWriteSerializer, serializers.py:47-51), então basta um erro de cadastro ou a desvinculação de um cliente para a conta virar uma chave-mestra silenciosa.

**Impacto**

Um único usuário mal cadastrado enxerga e EXPORTA o parque, as medições, as anomalias, as ordens de serviço e o histórico de manutenção de todos os clientes da plataforma, em planilha pronta. Como o próprio módulo oferece download em CSV/XLSX/PDF, o vazamento é de base inteira, não de registro isolado.

**Sugestão de correção**

Fazer _escopo devolver qs.none() quando user.is_cliente e não há cliente_id, alinhando com coletas/views.py:49-53. Melhor ainda: extrair as três cópias para uma única função em apps/core e importá-la nos três módulos, para a regra não poder divergir de novo.

**Critérios de aceite**

- [ ] _escopo devolve queryset vazio para usuário cliente sem cliente_id
- [ ] Teste: usuário CLIENTE_* sem vínculo recebe 0 linhas em cada um dos 5 relatórios abertos
- [ ] Filtro de tenant unificado num único helper compartilhado
- [ ] apps/servicos/views.py e apps/servicos/atividades.py migrados para o mesmo helper (ver A-04)

```

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

```

# [Segurança] Higiene de controle de acesso: filtro de tenant duplicado e /api/empresas/ legível por qualquer conta

**Labels sugeridas:** `security`, `severidade:baixa`

**Achado(s) da auditoria:** A-04, A-05

> Agrupa dois achados de baixa severidade e mesmo tema (permissão/escopo declarados fora do padrão do projeto). Nenhum dos dois é explorável para vazamento entre inquilinos hoje; ambos são dívida técnica de segurança que barateia a próxima regressão.

---

### A-04 – Duas cópias do filtro de tenant divergem do padrão fail-closed

**Problema**

```

Ao contrário de A-01, aqui NÃO há vazamento hoje: cliente_id None vira "cliente_id IS NULL" e ServicoCampo.cliente (servicos/models.py:48) e Carregamento.cliente são NOT NULL, então o resultado é vazio por acidente, não por desenho. O risco é de manutenção: qualquer futura coluna de tenant que aceite NULL, ou a reutilização deste helper com outro campo_cliente, transforma o acidente em vazamento.

****Evidência****

Arquivos: `backend/apps/servicos/views.py:26-30`, `backend/apps/servicos/atividades.py:96-97`

```
```python
backend/apps/servicos/views.py
26: def escopo_cliente(qs, user, campo_cliente="cliente"):
27: """Perfis cliente só enxergam os próprios serviços (Portal - item 2.7)."""
28: if user.is_cliente:
29: return qs.filter(**{campo_cliente: user.cliente_id}) # sem guarda de None
30: return qs
```

```
backend/apps/servicos/atividades.py
96: if self.request.user.is_cliente:
97: qs = qs.filter(cliente_id=self.request.user.cliente_id)
```
```

****Impacto****

Nenhum impacto explorável no código atual. Dívida técnica de segurança: a quarta cópia da mesma regra, com semântica diferente das outras três, aumenta a chance de a próxima alteração repetir o erro de A-01.

****Sugestão de correção****

Importar o helper único (ver A-01) em apps/servicos/views.py e apps/servicos/atividades.py e apagar as cópias locais.

A-05 – Tela de Prestadores é escondida por papel no front, mas /api/empresas/ é legível por qualquer conta

****Problema****

A rota /prestadores só aparece para usuário interno, mas ela é apenas um rótulo: a API por trás é /api/empresas/, cujo ViewSet herda a permissão padrão de BaseCadastroViewSet, que libera leitura a qualquer autenticado, e não declara campo_cliente. A escrita está corretamente bloqueada (PATCH/POST/DELETE exigem perfil interno).

****Evidência****

Arquivos: `frontend/src/lib/permissions.ts:142-143`, `frontend/src/lib/permissions.ts:220`,
`frontend/src/app/(admin)/prestadores/prestador-form.tsx:190-192`, `backend/apps/cadastros/views.py:142-145`,
`backend/apps/accounts/permissions.py:55-66`

```
```python
frontend/src/lib/permissions.ts
142: case "prestadores:gerenciar":
143: return interno;
220: { prefixo: "/prestadores", exige: "prestadores:gerenciar" },

frontend/src/app/(admin)/prestadores/prestador-form.tsx
190: await api(`/empresas/${prestadorId}/`, { method: "PATCH", body });

backend/apps/cadastros/views.py
142: class EmpresaViewSet(BaseCadastroViewSet): # herda InternoEditaClienteVisualiza
143: queryset = Empresa.objects.ativos() # campo_cliente = None (sem escopo)

backend/apps/accounts/permissions.py (InternoEditaClienteVisualiza)
59: if request.method in SAFE_METHODS:
60: return True # <- qualquer autenticado LÊ
```
```

Prova de conceito reproduzida (`manage.py test` com `APIClient`):

```
```text
Teste executado contra a API, usuario CLIENTE_MANUT / JUNIOR do Portal:
GET /api/empresas/ -> 200
{'nome': 'ThermoProActive Servicos Ltda', 'cnpj': '39.923.567/0001-75',
```

```
'email': 'financeiro@thermoproactive.com', 'telefone': '(19) 99999-0000',
'contato_gestor': 'Diretor Tecnico'}
POST /api/empresas/ (escrita) -> 403 (corretamente bloqueado)
```

**Impacto**

Exposição limitada: Empresa é o cadastro da própria CONTRATADA (razão social, CNPJ, inscrição estadual, endereço, e-mail, telefone e nome do gestor), dados que já aparecem no timbre dos relatórios entregues. Não há dado de outro inquilino aqui. Vale corrigir porque o endpoint também alimenta o cabeçalho de laudos e pode crescer com campos sensíveis.

**Sugestão de correção**

Trocar a permission_class de EmpresaViewSet para IsInterno (leitura e escrita), alinhando com o gate 'prestadores:gerenciar' do front. Se o Portal precisar do nome/logo do prestador para o cabeçalho, expor um endpoint público mínimo com apenas esses campos.

---

**Critérios de aceite**

_A-04_
- [ ] apps/servicos não define mais escopo_cliente próprio
- [ ] AtividadeCorretivaViewSet.get_queryset usa o helper compartilhado
- [ ] Teste: usuário cliente sem vínculo recebe lista vazia em /api/servicos/ e /api/atividades-corretivas/

_A-05_
- [ ] GET /api/empresas/ como usuário CLIENTE_* devolve 403
- [ ] Cabeçalho de relatório no Portal continua funcionando (endpoint dedicado ou dado já embutido)
```

--- FIM ISSUE 6 ---